Projektbericht

Entwicklung einer Flugsteuerung und Fernsteuerung für einen Quadrocopter auf Basis des ESP32

EIT-F-500: Projektseminar 1

Sommersemester 2026

Technische Hochschule Mittelhessen (THM)

Campus Friedberg, Fachbereich Informationstechnik-Elektrotechnik-Mechatronik (IEM)

vorgelegt von

Mabroz Khan

Betreuung: Prof. Dr.-Ing. Sergej Kovalev

Inhaltsverzeichnis

Abbildungsverzeichnis ..... **III**

Tabellenverzeichnis ..... **IV**

Abkürzungsverzeichnis ..... **V**

1 Einleitung **1**

1.1 Kurzfassung 1

1.2 Projektziele und Anforderungen 1

2 Grundlagen **2**

2.1 Aufbau eines Quadrocopters 2

2.2 Flugphysik 2

2.3 Lageerfassung mit Inertialsensoren 4

2.4 PID-Regler und Kaskadenregelung 5

2.5 Kommunikationsprotokolle 5

3 Stand der Technik **7**

4 Konzept und Komponentenauswahl **8**

4.1 Lösungskonzept 8

4.2 Auslegung 8

4.3 Komponentenliste 9

5 Hardware der Drohne **11**

5.1 Rahmen 11

5.2 Verkabelung und Energieversorgung 13

6 Software der Flugsteuerung **15**

6.1 Softwarearchitektur 15

6.2 Lageschätzung und Kalibrierung 15

6.3 Regelung und Motormischung 15

6.4 Sicherheitsfunktionen 17

7 Fernsteuerung und Funkkommunikation **19**

7.1 Hardware der Fernsteuerung 19

7.2 Software der Fernsteuerung 21

7.3 Funkübertragung mit ESP-NOW 22

8 Tests und Optimierung **23**

8.1 Vorgehen und Sicherheit 23

8.2 Inbetriebnahme der Komponenten 23

8.3 Iteration der Hardware 24

8.4 Einstellung der Regler 24

8.5 Fehlersuche anhand von Messdaten 25

9 Fazit und Ausblick **27**

9.1 Zusammenfassung 27

9.2 Herausforderungen und Erkenntnisse 27
9.3 Grenzen des Systems und Ausblick 28
Literaturverzeichnis 29
Hinweis zur Nutzung von KI 32
Anhang 33
A 3D-Modelle 33
B Quellcode der Flugsteuerung 33
C Quellcode der Fernsteuerung 33

Abbildungsverzeichnis

Abbildung 1: Anordnung und Drehrichtung der Motoren in X-Konfiguration 3
Abbildung 2: 3D-Modell des ursprünglichen Rahmens mit 134 mm Armlänge 11
Abbildung 3: 3D-Modell des überarbeiteten Rahmens mit 94 mm Armlänge 11
Abbildung 4: Aufgebauter Rahmen mit 134 mm Armlänge 12
Abbildung 5: Aufgebauter Rahmen mit 94 mm Armlänge 12
Abbildung 6: Schaltplan der Flugsteuerung 14
Abbildung 7: 3D-Modell des Gehäuses der Fernsteuerung 19
Abbildung 8: Aufgebaute Fernsteuerung 19
Abbildung 9: Schaltplan der Fernsteuerung 20
Abbildung 10: Anzeige der beiden Displays im Betrieb 22
Abbildung 11: MPU6050-Test auf dem Breadboard mit Neigungsvisualisierung 23
Abbildung 12: Testaufbau mit dem ursprünglichen Rahmen, Akku hier noch oben montiert 24
Abbildung 13: Standbild aus einem Testvideo, Akku unten montiert, Propeller durch die hohe Drehzahl kaum als solche erkennbar 25

Tabellenverzeichnis

Tabelle 1: Vergleich der beiden Aufbauten 9
Tabelle 2: Verwendete Komponenten 9
Tabelle 3: Beschaffungskosten 10
Tabelle 4: Verbindungen an der Flugsteuerung 14
Tabelle 5: Reglereinstellungen 16
Tabelle 6: Verbindungen an der Fernsteuerung 20

Abkürzungsverzeichnis

BEC	Battery Eliminator Circuit, Spannungswandler zur Versorgung der Elektronik aus dem Flugakku
BLDC	Brushless Direct Current, bürstenloser Gleichstrommotor
CAD	Computer-Aided Design
CCW	Counterclockwise, gegen den Uhrzeigersinn
CW	Clockwise, im Uhrzeigersinn
ESC	Electronic Speed Controller, elektronischer Drehzahlsteller
ESP-NOW	verbindungsloses WLAN-Protokoll von Espressif für die direkte Kommunikation zwischen ESP32-Modulen
GPIO	General Purpose Input/Output
GPS	Global Positioning System
I²C	Inter-Integrated Circuit
KV	Umdrehungen pro Minute je Volt, Drehzahlkonstante eines Motors
LiPo	Lithium-Polymer
MEMS	Micro-Electro-Mechanical Systems, mikroelektromechanische Systeme
OLED	Organic Light-Emitting Diode, organische Leuchtdiode
PID	Proportional-Integral-Derivative, proportional-integral-differenzierend
PLA	Polylactid, Kunststoff für den 3D-Druck
PWM	Pulsweitenmodulation
RC	Radio Control, Funkfernsteuerung
STL	Standard Tessellation Language, Dateiformat für 3D-Modelle
TWR	Thrust-to-Weight Ratio, Schub-Gewicht-Verhältnis
USB	Universal Serial Bus

1 Einleitung

1.1 Kurzfassung

Dieser Bericht beschreibt die Entwicklung eines Quadrocopters mit vollständig selbst programmierter Flugsteuerung sowie einer ebenfalls selbstgebauten Fernsteuerung, die über ESP-NOW mit der Drohne kommuniziert. Beide Geräte basieren auf einem ESP32; die Lageregelung erfolgt kaskadiert über PID-Regler auf Basis eines MPU6050 als einzigem Sensor. Der Bericht behandelt die Auswahl der Bauteile, den mechanischen und elektrischen Aufbau, die Software beider Systeme sowie die durchgeführten Tests, in deren Verlauf die Hardware einmal grundlegend überarbeitet wurde. Abschließend werden die Grenzen des aktuellen Systems und mögliche Erweiterungen eingeordnet.

1.2 Projektziele und Anforderungen

Das Thema wurde vom betreuenden Professor vorgeschlagen. Vorgegeben war, die Flugsteuerung selbst zu programmieren. Darüber hinaus gab es keine Vorgaben, weder zu den Bauteilen noch zu weiteren Anforderungen. Ziele und Umfang wurden deshalb selbst festgelegt und haben sich im Laufe des Projekts weiterentwickelt. Am Anfang stand eine Recherche zu den notwendigen Bauteilen und zum Aufbau von Drohnenrahmen.

Ziel ist ein flugfähiger Quadrocopter mit selbst entwickelter Flugsteuerung, die so gut regelt, wie es die verwendete Hardware zulässt. Statt eines fertigen Flight Controllers mit integrierter Sensorik übernimmt ein ESP32 diese Aufgabe. Er misst über einen MPU6050 die Lage der Drohne und regelt daraus die Drehzahlen der Motoren. Ein zweiter ESP32 bildet die Grundlage der ebenfalls selbst gebauten Fernsteuerung, die per Funk mit der Drohne verbunden ist.

Gleich zu Beginn wurde der Drohnentyp festgelegt. Gewählt wurde bewusst eine Drohne für den Einsatz im Freien. Ihre Bauteile sind leistungsstärker und teurer als bei einer kleinen Drohne für Innenräume, dafür bietet sie mehr Spielraum für spätere Erweiterungen. Die Abwägung beschreibt Kapitel 4.1 genauer.

2 Grundlagen

2.1 Aufbau eines Quadrocopters

Die Hauptkomponenten eines Quadrocopters sind der Rahmen, vier Motoren mit Propellern, die zugehörigen Motorregler, ein Akku, der Flight Controller mit seiner Sensorik und eine Funkverbindung zur manuellen Steuerung [1]. Der Rahmen trägt alle Bauteile und legt über die Länge der Arme fest, wie weit die Motoren vom Schwerpunkt entfernt sind.

Als Antrieb dienen in der Regel bürstenlose Gleichstrommotoren (BLDC) [1]. Sie kommen ohne Bürsten aus und werden stattdessen elektronisch kommutiert [2]. Der Rotor trägt Permanentmagnete, im Stator liegen meist drei Wicklungen [2]. Damit sich der Rotor dreht, müssen die Wicklungen in einer festen Reihenfolge bestromt werden, die von der aktuellen Lage des Rotors abhängt [2]. Als Vorteile gegenüber Motoren mit Bürsten nennt [2] unter anderem einen hohen Wirkungsgrad und ein hohes Drehmoment im Verhältnis zur Baugröße, was bei begrenztem Platz und Gewicht von Vorteil ist.

Diese Ansteuerung übernimmt der Electronic Speed Controller (ESC). Er setzt das Steuersignal des Flight Controllers in die passende Leistung für den Motor um [1].

Der Flight Controller ist die Hardware, auf der die Software zur Flugsteuerung läuft. Er ist mit Sensoren verbunden, aus deren Daten die Software den Zustand des Fluggeräts bestimmt [1]. Die Flugsteuerungssoftware PX4 setzt zum Beispiel mindestens ein Gyroskop, einen Beschleunigungssensor, ein Magnetometer und ein Barometer voraus. Bei Flight Controllern der Pixhawk-Serie sind diese Sensoren bereits eingebaut [1].

Die Energie liefert meist ein Lithium-Polymer-Akku (LiPo) [1]. Eine LiPo-Zelle hat eine Nennspannung von 3,7 V und ist bei 4,2 V voll geladen [3]. Für höhere Spannungen werden mehrere Zellen in Reihe geschaltet, deren Anzahl mit dem Buchstaben S angegeben wird [4]. Ein 3S-Akku besteht demnach aus drei Zellen und hat eine Nennspannung von 11,1 V, voll geladen 12,6 V.

Gesteuert wird der Quadrocopter über eine Fernsteuerung, deren Befehle per Funk an den Flight Controller gehen [1].

2.2 Flugphysik

Jeder Rotor erzeugt eine Schubkraft entlang seiner Drehachse, die mit dem Quadrat der Rotordrehzahl wächst [5]:

$$F_i = k \cdot \omega_i^2 \tag{2.1}$$

Dabei ist k die Schubkonstante. Die vier Einzelkräfte ergeben zusammen den Gesamtschub [5]. Steht der Quadrocopter waagerecht, beschleunigt er nach oben, sobald der Gesamtschub die Gewichtskraft übersteigt [5]. Für den Schwebeflug gilt deshalb

$$F_1 + F_2 + F_3 + F_4 = m \cdot g \tag{2.2}$$

Um steigen und gleichzeitig Lageänderungen ausregeln zu können, muss der maximal mögliche Schub über der Gewichtskraft liegen. Das Verhältnis beider Größen wird als Schub-Gewicht-Verhältnis bezeichnet:

$$TWR = \frac{F_{max}}{m \cdot g} \tag{2.3}$$

Die Lage des Quadrocopters wird mit drei Winkeln beschrieben, Roll um die Längsachse x, Pitch um die Querachse y und Yaw um die Hochachse z [5]. Ein Quadrocopter hat sechs Freiheitsgrade, lässt

sich aber nur über die vier Rotordrehzahlen steuern [5]. Seitliche Bewegungen entstehen deshalb nur indirekt, indem sich die Drohne neigt [6].

Roll und Pitch werden über Schubunterschiede zwischen gegenüberliegenden Seiten gesteuert [6]. Ein Schubunterschied ΔF erzeugt über den Hebelarm l, den Abstand zwischen Rotor und Schwerpunkt, ein Drehmoment [5]:

$$M = l \cdot \Delta F \tag{2.4}$$

Bei der X-Anordnung sind die Arme um 45° gegenüber den Achsen x und y gedreht. Für Roll und Pitch wirken dann jeweils zwei Rotoren gemeinsam, zum Beispiel beide linken gegen beide rechten. Der wirksame Hebelarm ist dabei kleiner als die Armlänge und beträgt l · sin 45°.

Neben dem Schub erzeugt jeder Rotor ein Reaktionsmoment um seine Drehachse, das ebenfalls mit dem Quadrat der Drehzahl wächst [5]:

$$M_i = b \cdot \omega_i^2 \tag{2.5}$$

Es wirkt entgegen der Drehrichtung des Rotors auf den Rahmen. Damit sich diese Momente im Schwebeflug aufheben, besteht ein Quadrocopter aus zwei Rotorpaaren mit entgegengesetzter Drehrichtung [6]. Bei der X-Anordnung drehen sich die diagonal gegenüberliegenden Rotoren in dieselbe Richtung, wie Abbildung 1 zeigt. Die Drehung um die Hochachse wird über den Unterschied der Reaktionsmomente beider Paare gesteuert [6]. Laufen die Rotoren eines Paares schneller und die des anderen langsamer, dreht sich der Quadrocopter, während der Gesamtschub gleich bleiben kann [5].

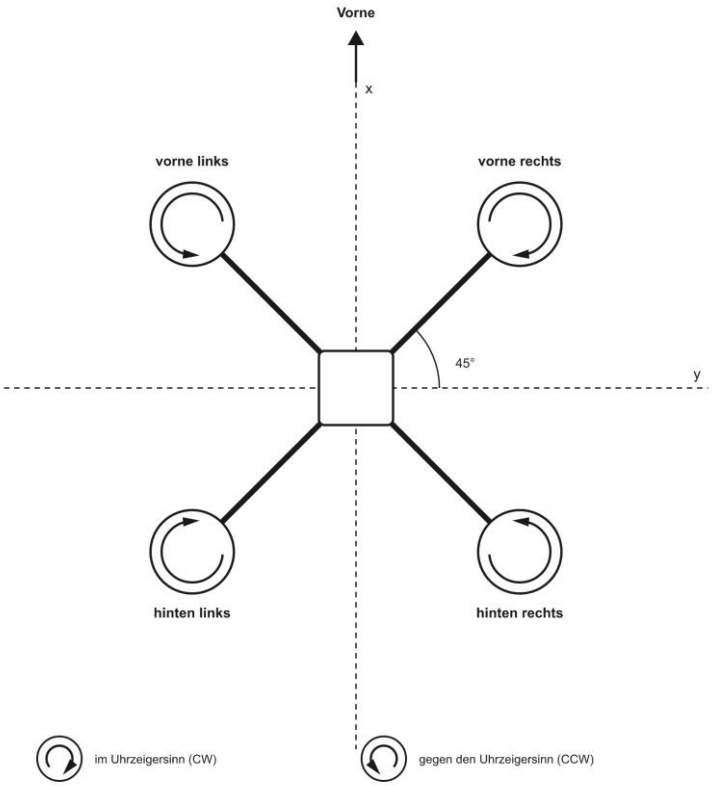


Abbildung 1: Anordnung und Drehrichtung der Motoren in X-Konfiguration

Wie stark ein Körper auf ein Drehmoment reagiert, bestimmt sein Trägheitsmoment J. Für ein System aus Punktmassen gilt [7]

$$J = \sum_j m_j \cdot r_j^2 \tag{2.6}$$

und für die Winkelbeschleunigung α [7]

$$M = J \cdot \alpha \tag{2.7}$$

Masse weit von der Drehachse erhöht das Trägheitsmoment stärker als Masse nahe der Achse [7]. Betrachtet man die Motoren vereinfacht als Punktmassen an den Armenden, wächst das Trägheitsmoment mit dem Quadrat der Armlänge, das Drehmoment aus Gleichung (2.4) dagegen nur linear. Kürzere Arme führen deshalb bei gleichem Schubunterschied zu einer größeren Winkelbeschleunigung.

2.3 Lageerfassung mit Inertialsensoren

Zur Lageerfassung dienen Inertialsensoren, die Drehrate und Beschleunigung messen. Der MPU6050 vereint ein Gyroskop und einen Beschleunigungssensor mit jeweils drei Achsen in MEMS-Technik in einem Gehäuse von 4 × 4 × 0,9 mm [8]. Die Messbereiche sind einstellbar, beim Gyroskop von ±250 °/s bis ±2000 °/s, beim Beschleunigungssensor von ±2 g bis ±16 g [8].

Ein MEMS-Beschleunigungssensor besteht aus einer beweglichen Masse, die an Federn aufgehängt ist. Ihre Auslenkung wird über die Änderung einer Kapazität gemessen [9]. Der Sensor erfasst dabei die Differenz aus eigener Beschleunigung und Erdbeschleunigung [9]. Ruht er, zeigt der Messwert nur die Richtung der Schwerkraft an, und daraus lassen sich Roll- und Pitchwinkel berechnen [9]:

$$\tan \varphi = \frac{a_y}{a_z} \tag{2.8}$$

$$\tan \theta = \frac{-a_x}{\sqrt{a_y^2 + a_z^2}} \tag{2.9}$$

Diese Rechnung setzt voraus, dass keine weitere Beschleunigung auf den Sensor wirkt. Beschleunigungen durch Bewegung verfälschen den berechneten Winkel, und das Signal ist verrauscht [9], [10]. Eine Drehung um die Schwerkraftachse ändert das Messsignal nicht. Den Gierwinkel kann ein Beschleunigungssensor deshalb grundsätzlich nicht bestimmen, dafür wäre zusätzlich ein Magnetometer nötig [9].

Das Gyroskop misst die Drehrate ω. Den Winkel erhält man durch Aufsummieren über die Zeit [10]:

$$\theta_k = \theta_{k-1} + \omega \cdot \Delta t \tag{2.10}$$

Das Ergebnis reagiert schnell und wird von Beschleunigungen nicht beeinflusst. Zeigt das Gyroskop im Stillstand aber nicht exakt null an, addiert sich dieser kleine Fehler bei jedem Schritt, und der berechnete Winkel driftet immer weiter weg [10].

Der Komplementärfilter verbindet beide Messungen. Er vertraut kurzfristig dem Gyroskop und langfristig dem Beschleunigungssensor [10]:

$$\theta_k = a \cdot (\theta_{k-1} + \omega \cdot \Delta t) + (1 - a) \cdot \theta_{acc} \tag{2.11}$$

Der Faktor a legt zusammen mit der Abtastzeit Δt die Zeitkonstante τ des Filters fest [10]:

$$\tau = \frac{a \cdot \Delta t}{1 - a} \tag{2.12}$$

Änderungen, die kürzer als τ sind, folgt der Filter über das Gyroskop, auf längere Sicht setzt sich der Beschleunigungssensor durch [10]. Ein kleines τ gleicht die Drift schneller aus, lässt aber mehr Störungen durch Querbeschleunigungen durch. Einen passenden Wert findet man letztlich im Versuch [10].

Ein konstanter Messfehler des Gyroskops erzeugt dabei eine bleibende Winkelabweichung, die dem Fehler multipliziert mit τ entspricht [10]. Der Kalman-Filter bezieht zusätzlich ein physikalisches Modell ein, ist aber deutlich aufwendiger umzusetzen [10].

2.4 PID-Regler und Kaskadenregelung

Ein Regler vergleicht laufend einen Sollwert r mit dem gemessenen Istwert y. Aus der Regelabweichung e = r − y berechnet er die Stellgröße u [11]. Am weitesten verbreitet ist der PID-Regler, dessen Stellgröße sich aus drei Anteilen zusammensetzt [11]:

$$u(t) = k_p \cdot e(t) + k_i \cdot \int_0^t e(t')\,dt' + k_d \cdot \frac{de(t)}{dt} \tag{2.13}$$

Der Proportionalanteil reagiert direkt auf die Abweichung. Mit größerem k_p wird die bleibende Regelabweichung kleiner, das System schwingt aber stärker [11]. Der Integralanteil summiert die Abweichung über die Zeit auf und beseitigt dadurch die bleibende Regelabweichung. Ein zu großes k_i führt zu Schwingungen und kann das System instabil machen [11]. Der Differentialanteil wirkt auf die Änderung der Abweichung und dämpft Schwingungen. Ist k_d zu groß, verschlechtert sich das Verhalten wieder [11]. Weil eine Ableitung hochfrequente Signale stark verstärkt, erzeugt schon geringes Messrauschen große Schwankungen in der Stellgröße. Der Differentialanteil wird deshalb in der Praxis auf ein tiefpassgefiltertes Signal angewendet [11].

Auf einem Mikrocontroller wird der Regler zeitdiskret umgesetzt. Er läuft periodisch mit der Abtastzeit h, liest in jedem Durchlauf den Messwert ein, berechnet die Stellgröße und gibt sie aus [11]. Das Integral wird dabei durch eine Summe ersetzt [11]:

$$I_{k+1} = I_k + k_i \cdot h \cdot e_k \tag{2.14}$$

Jedes Stellglied hat Grenzen, ein Motor zum Beispiel eine maximale Drehzahl. Erreicht die Stellgröße diese Grenze, bleibt sie dort unabhängig von der Regelabweichung stehen, und der Regelkreis ist praktisch unterbrochen. Der Integralanteil wächst in dieser Zeit weiter, weil die Abweichung bestehen bleibt. Kommt das System zurück in den Arbeitsbereich, muss der aufgelaufene Wert erst wieder abgebaut werden, und es entstehen große Ausschläge. Dieser Effekt heißt Integrator-Windup [11]. Als Gegenmaßnahme wird der Integralanteil begrenzt oder nur unter bestimmten Bedingungen weiter aufsummiert [11], [12].

Die Flugsteuerungssoftware PX4 verwendet für Multicopter eine Kaskadenregelung. Ein äußerer Lageregler mit reinem Proportionalanteil berechnet aus dem Sollwinkel eine Soll-Drehrate und gibt sie an einen inneren Drehratenregler weiter, der als PID-Regler ausgeführt ist [12].

2.5 Kommunikationsprotokolle

ESCs mit PWM-Eingang erhalten ihren Sollwert über die Pulsbreite. Sie liegt typischerweise zwischen 1000 µs für keine Leistung und 2000 µs für volle Leistung [13]. Die Wiederholrate hängt vom ESC ab und liegt üblicherweise zwischen 50 Hz und 490 Hz [13]. Das Verhältnis von Pulsbreite t_{Puls} zur Periodendauer T ist der Tastgrad:

$$D = \frac{t_{Puls}}{T} \tag{2.15}$$

Neben PWM gibt es schnellere Protokolle wie OneShot und DShot [13].

Der Sensor ist über den I²C-Bus angebunden. Er benötigt nur zwei Leitungen, die Datenleitung SDA und die Taktleitung SCL, die über Pull-up-Widerstände an der Versorgungsspannung liegen [14]. Jedes Gerät am Bus hat eine eindeutige Adresse. Der Controller startet eine Übertragung und erzeugt den Takt,

das angesprochene Gerät heißt Target [14]. Seit Revision 7 der Spezifikation ersetzen diese Begriffe die früheren Bezeichnungen Master und Slave [14]. Die Datenrate beträgt im Standard-Mode bis zu 100 kbit/s und im Fast-Mode bis zu 400 kbit/s [14]. Der MPU6050 arbeitet am Bus immer als Target, seine Adresse ist je nach Pegel am Anschluss AD0 entweder 0x68 oder 0x69 [8].

Für die Funkverbindung zwischen zwei ESP32 stellt Espressif das Protokoll ESP-NOW bereit. Es arbeitet verbindungslos, die Nutzdaten werden in einem herstellerspezifischen WLAN-Rahmen direkt von einem Gerät zum anderen übertragen [15]. Die Gegenstellen werden über ihre MAC-Adresse angesprochen. Ein Paket fasst in Version 1.0 bis zu 250 Byte Nutzdaten, in Version 2.0 bis zu 1470 Byte [15]. Senden und Empfangen werden über Callback-Funktionen gemeldet. Weil diese im WLAN-Task laufen, sollen darin keine längeren Operationen stattfinden [15].

3 Stand der Technik

Für die Flugsteuerung von Multicoptern gibt es etablierte Open-Source-Software. Bei der Recherche für dieses Projekt war Betaflight die wichtigste Anlaufstelle, und auch im weiteren Verlauf diente es als Orientierung. Betaflight ist eine Firmware für Flight Controller von Multicoptern und Flächenflugzeugen [16]. Sie läuft auf Flight Controllern mit STM32-Prozessoren der Reihen F4, G4, F7 und H7 und steht unter der Lizenz GPL v3 [16]. Zum Funktionsumfang gehören unter anderem die Einstellung der PID-Werte während des Flugs, umschaltbare Rate-Profile, eine Blackbox zur Aufzeichnung von Flugdaten und digitale Motorprotokolle wie DShot [16].

Auf autonome Flüge sind dagegen ArduPilot und PX4 ausgerichtet. ArduPilot ist Open-Source-Software, die auf unterschiedlicher Hardware läuft und neben Multicoptern unter anderem auch Flugzeuge, Landfahrzeuge und Unterwasserfahrzeuge steuert [17]. In autonomen Flugmodi fliegt ArduPilot vorab geplante Missionen ab [17]. PX4 ist ein Open-Source-Autopilot für Drohnen und andere autonome Fahrzeuge [18]. Dafür setzt PX4 neben Gyroskop und Beschleunigungssensor auch ein Magnetometer und ein Barometer voraus [1].

Fertige kommerzielle Drohnen nutzen zusätzliche Sensoren zur Positionsbestimmung. Die DJI Mini 4 Pro wiegt beim Start weniger als 249 g [19]. Sie empfängt die Satellitensysteme GPS, Galileo und BeiDou und besitzt Sichtsensoren nach vorne, hinten, zur Seite und nach unten sowie einen 3D-Infrarotsensor [19]. Mit Sichtpositionierung gibt der Hersteller eine Schwebegenauigkeit von ±0,1 m an, mit Satellitennavigation von ±0,5 m [19].

Näher an diesem Projekt liegt ESP-Drone von Espressif, eine Open-Source-Drohne auf Basis der Chips ESP32, ESP32-S2 und ESP32-S3, die über WLAN per App oder Gamepad gesteuert wird [20]. Der Hauptteil des Codes wurde aus dem Open-Source-Projekt Crazyflie übernommen [20]. Als Inertialsensor dient wie in diesem Projekt ein MPU6050 [21]. Neben dem Stabilisierungsmodus bietet ESP-Drone Modi zum Halten von Höhe und Position [20]. Für die Höhe ist das Druckmodul MS5611 vorgesehen, für die Position die Kombination aus dem optischen Flusssensor PMW3901 und dem Laser-Abstandssensor VL53L1X [21]. Außerdem lässt sich die Drohne per Joystick über ESP-NOW steuern [20].

Alle genannten Systeme bringen eine fertige Flugsteuerung mit. Für dieses Projekt war dagegen vorgegeben, die Flugsteuerung selbst zu programmieren. Deshalb wird keine dieser Firmwares übernommen, sondern eine eigene Flugsteuerung für den ESP32 geschrieben, wobei Betaflight als Orientierung dient. ESP-Drone zeigt, dass ein ESP32 mit einem MPU6050 grundsätzlich als Flight Controller ausreicht. Die dort vorgesehenen Sensoren für Höhe und Position deuten zugleich an, welche Erweiterungen für ein stabileres Schweben nötig wären.

4 Konzept und Komponentenauswahl

4.1 Lösungskonzept

Für den Mikrocontroller fiel die Wahl auf den ESP32. Ausschlaggebend waren der niedrige Preis, die bereits vorhandene eigene Erfahrung im Umgang mit dem Chip und sein großer Funktionsumfang mit vielen GPIO-Pins, eingebautem WLAN und Bluetooth sowie einer externen Antenne bei der verwendeten Variante [22]. Als Inertialsensor wurde der MPU6050 gewählt, ein weit verbreiteter Standardsensor für diesen Einsatzzweck.

Statt einer fertigen RC-Fernsteuerung wurde eine eigene Fernsteuerung auf Basis eines zweiten ESP32 entwickelt. Eine kommerzielle RC-Anlage samt passendem Empfänger ist vergleichsweise teuer. Da der ESP32 mit ESP-NOW bereits eine ausreichend reichweitenstarke Funkverbindung mitbringt, war kein zusätzliches Empfängermodul nötig. Hinzu kam der Wunsch, mit dem Bau einer eigenen Fernsteuerung einen größeren Eigenanteil an der Hardware zu übernehmen. Die Stromversorgung der Fernsteuerung übernimmt eine 18650-Zelle aus einer ausgeschlachteten Powerbank, die ohnehin vorhanden war.

Der Rahmen wurde selbst konstruiert und im 3D-Druck aus PLA gefertigt. Bei der Auswahl von Motoren, Motorreglern und Propellern standen Preis und Leistung im Vordergrund. Die Motoren mussten stark genug sein, um die Drohne sicher abheben zu lassen, ohne unnötig teuer zu sein, denn sie gehören zu den kostenintensivsten Bauteilen einer Drohne. Der Motorregler wurde passend zur Stromaufnahme der Motoren gewählt, die Propellergröße richtete sich nach der Motorspezifikation.

Als Energiequelle diente zunächst ein 2S-LiPo-Akku, da die gewählten Motoren laut Hersteller für 2S bis 3S ausgelegt sind [23]. Im Betrieb zeigte sich jedoch, dass die Spannung eines 2S-Akkus unter Last schnell unter 7 V fiel, was für ein zuverlässiges Abheben nicht mehr ausreichte. Die Drohne benötigte dabei rund 70 % der Gasstellung. Der Wechsel auf einen 3S-Akku behob das Problem, denn dessen Spannung liegt auch unter Last nicht unter 11 V, sodass die Motoren durchgehend genügend Leistung liefern. Dadurch reichten schon 30 bis 40 % der Gasstellung zum Abheben. Weil der neue Akku eine geringere Kapazität hat, fällt die Flugzeit dafür kürzer aus.

Die ursprüngliche Drohne mit dem größeren Rahmen nutzte dieselben Motoren, denselben Motorregler und dieselben Propeller wie die spätere Version. Verändert wurden ausschließlich Rahmen und Akku.

4.2 Auslegung

Für den verwendeten Motortyp lag keine Schubkurve des Herstellers vor, und eine eigene Messung des Standschubs war im Rahmen dieses Projekts nicht möglich. Das Schub-Gewicht-Verhältnis nach Gleichung (2.3) konnte deshalb nicht rechnerisch bestimmt werden. Nach Gleichung (2.2) muss der Gesamtschub mindestens die Gewichtskraft erreichen, und für Steigflug und Lageregelung ist zusätzliche Reserve nötig. Ob diese Reserve vorhanden ist, wurde stattdessen im Flugversuch geprüft. Die Drohne hebt mit dem 3S-Akku bereits bei einer Gasstellung von 30 bis 40 % ab, wie in Kapitel 8 beschrieben. Der verfügbare Schub liegt damit deutlich über der Gewichtskraft.

Die beiden Aufbauten unterscheiden sich in Gewicht, Armlänge und Energieversorgung, wie Tabelle 1 zeigt:

Tabelle 1: Vergleich der beiden Aufbauten

Merkmal	Rahmen groß, 2S	Rahmen klein, 3S
Gesamtgewicht mit Akku	230 g	167,5 g
Armlänge, Motor bis Rahmenmitte	134 mm	94 mm
Akkuspannung, voll geladen	8,4 V	12,6 V
Akkuspannung unter Last	teilweise unter 7 V	mindestens 11 V
Gasstellung beim Abheben	etwa 60 bis 70 %	etwa 30 bis 40 %

Die Armlänge wurde jeweils von der Position des Motors bis zur Mitte des Rahmens gemessen und entspricht damit dem Hebelarm l aus Gleichung (2.4). Nach Gleichung (2.6) wächst das Trägheitsmoment mit dem Quadrat dieser Länge, das Drehmoment aus Gleichung (2.4) dagegen nur linear. Die kürzeren Arme des neuen Rahmens erhöhen deshalb die erreichbare Winkelbeschleunigung zusätzlich zum Gewinn durch die höhere Akkuspannung.

4.3 Komponentenliste

Tabelle 2 listet alle verwendeten Komponenten mit ihren Kenndaten auf, Tabelle 3 die angefallenen Beschaffungskosten.

Tabelle 2: Verwendete Komponenten

Komponente	Typ oder Modell	Anzahl	Kenndaten	Quelle
Mikrocontroller	ESP32 DevKitC, WROOM-32U	2	externe 2,4-GHz-Antenne, USB-C, je einer in Drohne und Fernsteuerung	[22]
Inertialsensor	MPU6050 auf GY-521-Board	1	3-Achsen-Gyroskop und 3-Achsen-Beschleunigungssensor, I²C	[24]
Motor	YSIDO 1204, 6000 KV	4	2S bis 3S	[23]
Motorregler	HAKRC BLHeli_S, 4-in-1	1	15 A Dauerstrom, 20 A maximal, 2S bis 4S, ohne BEC, 4,8 g	[25]
Propeller	Gemfan 3020, 3 Blatt, Polycarbonat	4	Durchmesser 76,5 mm, 1,3 g, je zwei CW und CCW	[26]
Akku Drohne	Tattu R-Line 3S	1	650 mAh, 11,1 V, 95C, 60 g, XT30	[27]
Akku Drohne, ursprünglich	Teranty 2S	1	1500 mAh, 7,4 V, 45C, 60 g	[28]
Spannungswandler Drohne	Micro BEC	1	Eingang 6 bis 28 V, Ausgang 5 V und 3 A, 1,4 g	[29]
Spannungsteiler Drohne	Widerstände	2	31,6 kΩ und 9,86 kΩ	
Display	GERUI OLED, 0,91 Zoll	2	128 x 32 Pixel, I²C, 3,3 bis 5 V	[30]
Joystick	KY-023	2	zweiachsig, mit Taster	[31]
Akku Fernsteuerung	18650-Zelle aus einer Powerbank	1	3,7 V nominal, wiederverwendet	

Komponente	Typ oder Modell	Anzahl	Kenndaten	Quelle
Lademodul und Step-up-Wandler	VISSQH-Set	je 1	Laden über USB-C, Ausgangs-spannung 5 V	[32]
Spannungsteiler Fernsteuerung	Widerstände	2	je 10 kΩ	
Rahmen und Gehäuse	Eigenkonstruktion, 3D-Druck		PLA	

Tabelle 3: Beschaffungskosten

Position	Gekaufte Menge	Kosten
Motoren YSIDO 1204, 6000 KV	4	30,00 €
Motorregler HAKRC 4-in-1, 15 A	1	20,80 €
Propeller Gemfan 3020	4	3,09 €
Akku Tattu 3S, 650 mAh	1	26,00 €
Akkus Teranty 2S, 1500 mAh	2	16,00 €
ESP32 DevKitC WROOM-32U	2	13,69 €
Micro BEC	1	2,71 €
Joysticks KY-023	2	2,30 €
OLED-Displays 0,91 Zoll	3	9,00 €
Set aus Lademodul, Step-up-Wandler und Batteriehalter	je 5	8,00 €
Summe		131,59 €

Die Aufstellung nennt die tatsächlichen Beschaffungskosten einschließlich Reserveteilen, etwa drei Displays bei zwei verbauten. Nicht enthalten sind das Filament für den 3D-Druck sowie Kleinteile wie Widerstände, Kondensatoren und Kabel, die bereits vorhanden waren und im Centbereich liegen. Die 18650-Zelle stammt aus einer ausgeschlachteten Powerbank und verursachte keine Kosten.

5 Hardware der Drohne

5.1 Rahmen

Der Rahmen wurde selbst konstruiert und im 3D-Druck aus PLA gefertigt. Er besteht aus einer rechteckigen Mittelplatte, auf der Flugsteuerung und ESC übereinander montiert sind, sowie vier Armen in X-Anordnung mit den Motorhalterungen an den Enden (Abbildung 2 und Abbildung 3 zeigen die CAD-Modelle der beiden Rahmengrößen).

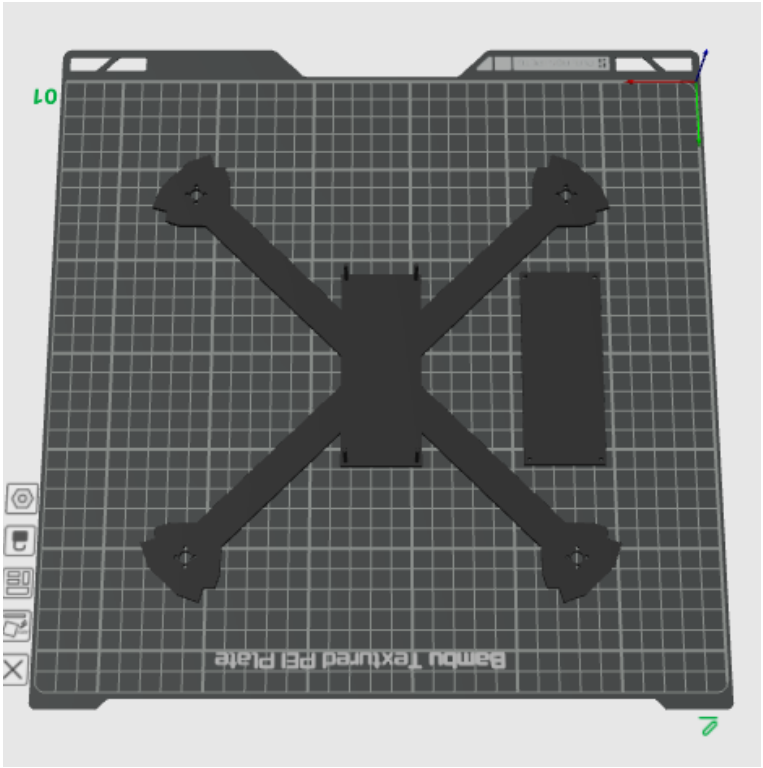


Abbildung 2: 3D-Modell des ursprünglichen Rahmens mit 134 mm Armlänge

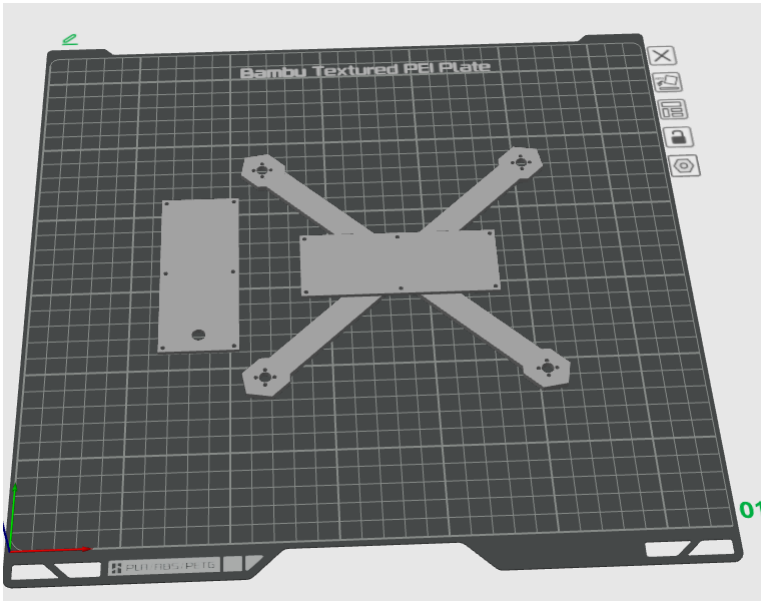


Abbildung 3: 3D-Modell des überarbeiteten Rahmens mit 94 mm Armlänge

Die Armlänge des überarbeiteten Rahmens wurde auf 94 mm festgelegt. Das ist die kürzeste Länge, bei der sich die 76,5 mm großen Gemfan-3020-Propeller in der X-Anordnung weder untereinander noch mit dem Rahmen berühren; das wurde vor dem Druck im CAD-Modell geprüft. Wie in Kapitel 4.2 beschrieben, sank durch die kürzeren Arme zusammen mit dem 3S-Akku das Gesamtgewicht von 230 g auf 167,5 g (Abbildung 4 und Abbildung 5 zeigen die aufgebauten Rahmen im Vergleich).

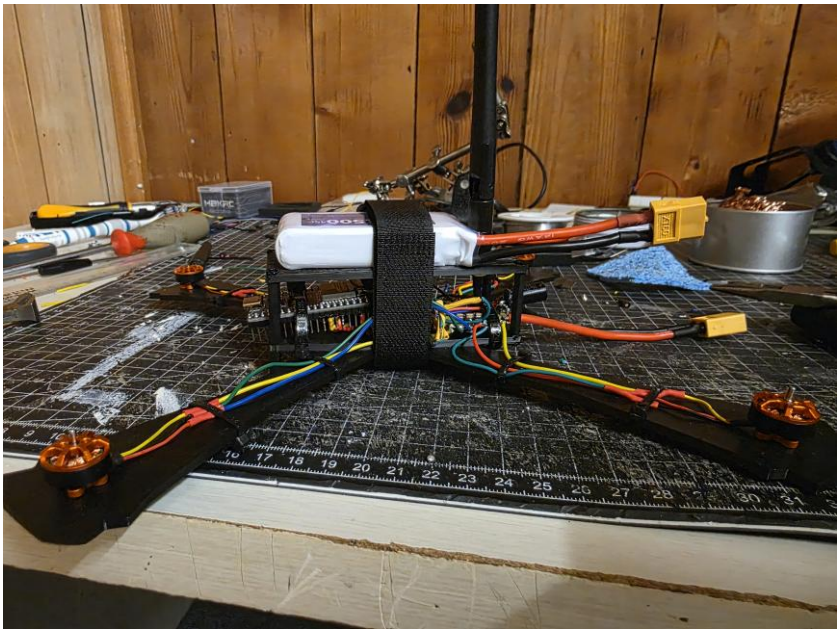


Abbildung 4: Aufgebauter Rahmen mit 134 mm Armlänge

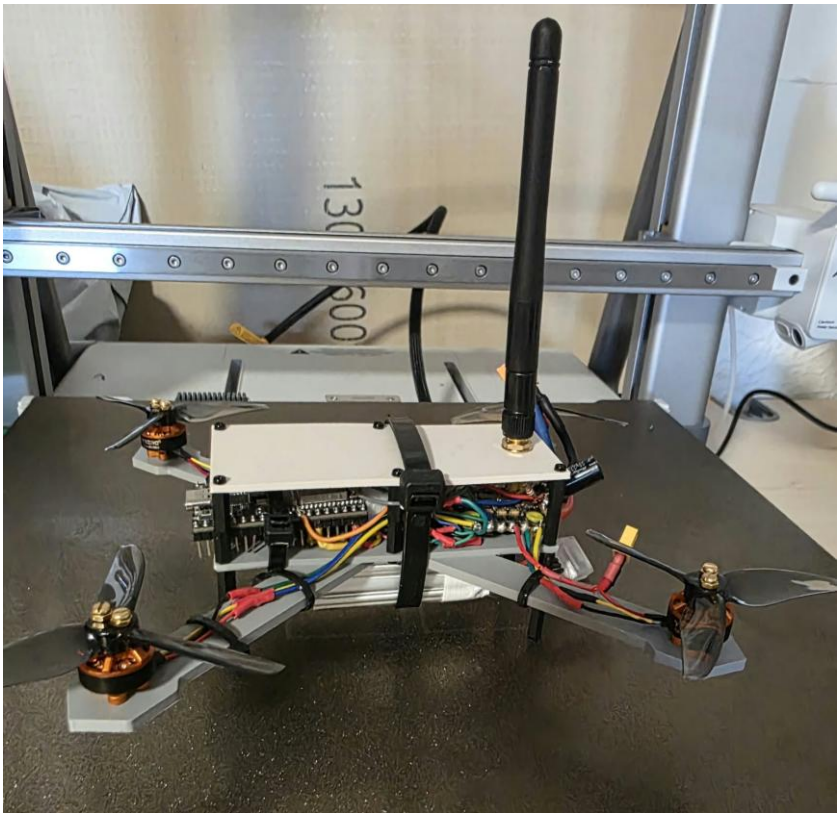


Abbildung 5: Aufgebauter Rahmen mit 94 mm Armlänge

Flugsteuerung, ESC und Sensor sind mit Kabelbindern am Rahmen befestigt statt verschraubt oder verklebt. Das ließ sich während der Entwicklung schnell lösen und wieder anpassen, etwa beim Wechsel zwischen den beiden Rahmengrößen. An den vier Ecken des Rahmens sitzen zusätzlich kurze M2-Gewindestangen mit Muttern als Standfüße, damit die Drohne möglichst eben auf dem Boden steht.

5.2 Verkabelung und Energieversorgung

Der Akku ist direkt an den Eingang des 4-in-1-ESCs gelötet. An derselben Stelle sitzt ein Kondensator mit 470 µF, der Rückstromspitzen abfängt: Werden die Motoren abgeschaltet, drehen sie kurz nach und speisen dabei Strom zurück, den der Kondensator aufnimmt. Von diesem Versorgungspunkt zweigen sowohl die Zuleitung zum Micro BEC als auch der Spannungsteiler zur Messung der Akkuspannung ab. Der Spannungsteiler führt zum Analogeingang GPIO 34 des ESP32.

Der Micro BEC wandelt die Akkuspannung auf 5 V und versorgt darüber den ESP32. Von dessen 3,3-V- und Masseanschluss führen zwei weitere Leitungen zum MPU6050, ergänzt um die beiden I²C-Leitungen SDA und SCL. Die externe Antenne ist direkt auf den dafür vorgesehenen Anschluss des ESP32-WROOM-32U-Moduls gesteckt.

Die vier Signalleitungen des ESCs sind mit den GPIOs 32, 25, 27 und 26 verbunden, die im Code den Motoren vorne links, vorne rechts, hinten links und hinten rechts zugeordnet sind. Der Kabelbaum des ESCs enthält eine fünfte Leitung, die für eine BEC-Ausgangsspannung vorgesehen wäre. Da hier ein separater Micro BEC verwendet wird, blieb dieser Anschluss ungenutzt und wurde abgetrennt.

Bei der Verkabelung der Motorphasen wurde auf die Drehrichtung geachtet: Vorne links und hinten rechts sind in derselben Reihenfolge angeschlossen, hinten links und vorne rechts ebenfalls untereinander gleich, aber gegenüber dem ersten Paar vertauscht. Dadurch drehen sich die beiden diagonalen Motorpaare gegenläufig, wie in Kapitel 2.2 beschrieben. Abbildung 6 gibt den vollständigen Schaltplan wieder, Tabelle 4 listet die einzelnen Verbindungen auf.

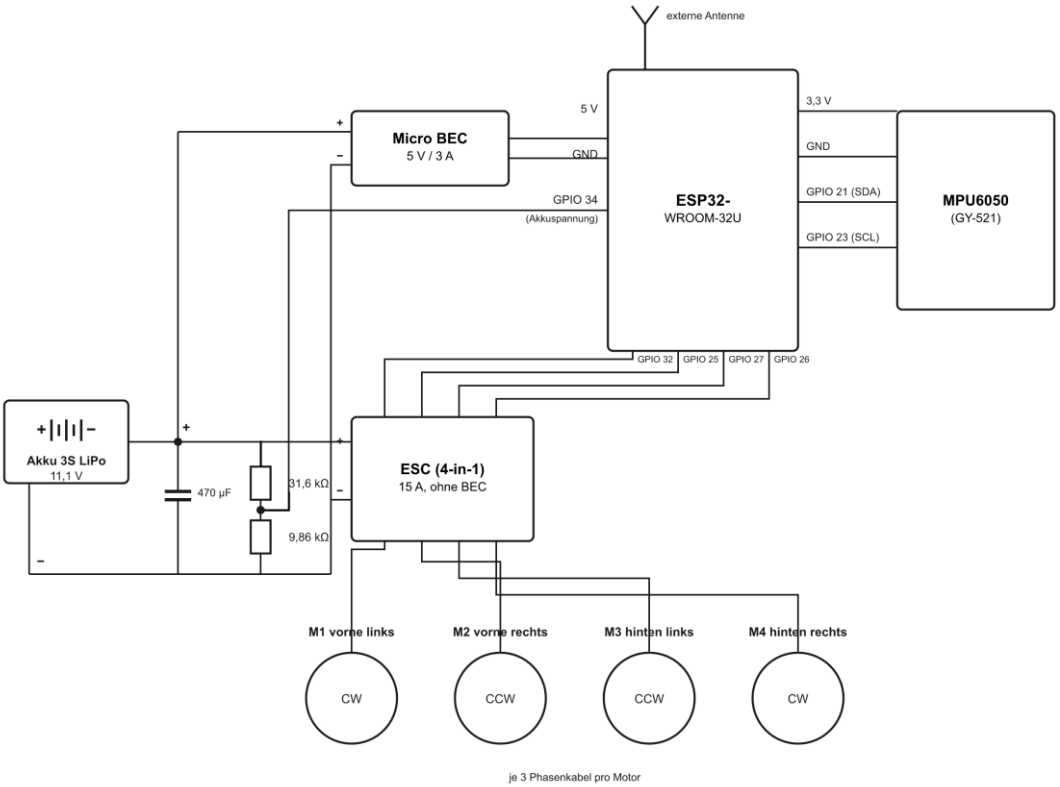


Abbildung 6: Schaltplan der Flugsteuerung

Tabelle 4: Verbindungen an der Flugsteuerung

Von	Nach	Funktion
Akku (+ / −)	Eingang ESC	Hauptversorgung
Kondensator, 470 µF	parallel zum Akku am ESC-Eingang	fängt Rückstromspitzen ab, wenn die Motoren beim Abschalten nachdrehen
ESC-Eingang	Micro BEC, Eingang	Versorgung des BEC
ESC-Eingang	Spannungsteiler (31,6 kΩ / 9,86 kΩ)	Messpfad für die Akkuspannung
Spannungsteiler, Mittelpunkt	ESP32 GPIO 34	Analogeingang Akkuspannung
Micro BEC, 5 V / GND	ESP32, 5 V / GND	Versorgung des ESP32
ESP32, 3,3 V / GND	MPU6050, VCC / GND	Versorgung des Sensors
ESP32 GPIO 21	MPU6050, SDA	I²C-Datenleitung
ESP32 GPIO 23	MPU6050, SCL	I²C-Taktleitung
ESC, Signal Motor 1	ESP32 GPIO 32	PWM vorne links, 400 Hz
ESC, Signal Motor 2	ESP32 GPIO 25	PWM vorne rechts, 400 Hz
ESC, Signal Motor 3	ESP32 GPIO 27	PWM hinten links, 400 Hz
ESC, Signal Motor 4	ESP32 GPIO 26	PWM hinten rechts, 400 Hz
ESC	je 3 Phasenkabel pro Motor	Motoransteuerung

6 Software der Flugsteuerung

6.1 Softwarearchitektur

In einer frühen Fassung lief die gesamte Flugsteuerung in einer einzigen Funktion ohne Aufteilung. Das führte zu unregelmäßigen Zeitabständen zwischen den Regelzyklen, weil serielle Ausgaben, Sensorabfrage und Motoransteuerung sich gegenseitig verzögerten. Die aktuelle Fassung verteilt die Arbeit deshalb auf FreeRTOS-Tasks: Der Regelkreis läuft mit hoher Priorität auf Kern 1, angestoßen durch einen Hardware-Timer im 1-kHz-Takt, während Akkumessung und Telemetrie mit niedrigerer Priorität auf Kern 0 laufen.

Weil Funkempfang und Regelkreis dadurch auf verschiedenen Kernen laufen, müssen die empfangenen Stickwerte gegen gleichzeitigen Zugriff abgesichert werden. Die Empfangsroutine verwirft zunächst jedes Paket, dessen Länge nicht exakt der erwarteten Datenstruktur entspricht. Der gültige Datensatz wird anschließend innerhalb einer kritischen Sektion in einem Stück übernommen, und der Regelkreis liest ihn auf demselben Weg wieder aus. Ohne diese Absicherung könnte der Regler einen Datensatz auswerten, der gerade erst zur Hälfte überschrieben wurde, und für einen Zyklus mit einer Mischung aus zwei Paketen rechnen.

6.2 Lageschätzung und Kalibrierung

Lageschätzung und Kalibrierung folgen dem Verfahren aus Kapitel 2.3, mit der auf eine Sekunde erhöhten Zeitkonstante des Komplementärfilters. Ihre Notwendigkeit ergab sich aus einer eigenen Beobachtung während der Tests: Die aus dem Beschleunigungssensor allein berechnete Lage zeigte starkes Rauschen und driftete bei Vibration deutlich. Die anschließende Recherche zu Lösungen für dieses Problem führte zum Komplementärfilter.

6.3 Regelung und Motormischung

Die Regelung ist als Kaskade aus zwei Reglern pro Achse aufgebaut. Ein äußerer Winkelregler vergleicht den aus den Sticks abgeleiteten Zielwinkel mit dem aktuellen Winkel aus Kapitel 6.2 und berechnet daraus eine Soll-Drehrate mit KP_ANGLE = 4,0:

$$rate_sp = KP_ANGLE \cdot (ziel_winkel - angle) \tag{6.1}$$

Diese Soll-Drehrate wird anschließend auf ±250 °/s begrenzt. Ohne diese Grenze könnte ein großer Winkelfehler eine Drehrate anfordern, die deutlich über dem liegt, was die Drohne überhaupt erreichen kann; die Begrenzung hält die Führungsgröße des inneren Reglers damit in einem realistisch erreichbaren Bereich.

Der Zielwinkel für Roll und Pitch sowie die Stick-Drehrate für Yaw werden aus dem rohen Stickwert in mehreren Schritten abgeleitet. Zunächst absorbiert ein Totband von 45 Einheiten um die Mittelstellung 1500 das Rauschen der Potentiometer, ähnlich wie beim Gas-Stick der Fernsteuerung in Kapitel 7.2. Der verbleibende Ausschlag wird auf den Bereich −1 bis 1 normiert und anschließend über eine kubische Kennlinie geführt, die zu 60 % aus dem Kubus und zu 40 % aus dem linearen Wert besteht. Das macht die Steuerung in Stickmittelnähe feinfühliger, ohne den vollen Ausschlag am Stickende zu verlieren. Der so bearbeitete Wert wird für Roll und Pitch auf ±20° Zielwinkel skaliert, für Yaw auf ±80 °/s Stick-Drehrate.

Für Yaw gilt, sobald die aus dem Stick abgeleitete Soll-Drehrate 2 °/s überschreitet, diese Drehrate direkt als Vorgabe, während im Hintergrund der Gierwinkel durch Aufsummieren der Gyroskop-Drehrate mitgeführt wird. Lässt der Pilot den Stick los, übernimmt statt der Drehrate eine Richtungshaltung: Ein P-Regler mit KP_YAW_ANGLE = 3,0 zieht die Drohne auf die zuletzt aktiv angesteuerte Richtung zurück, begrenzt auf höchstens 120 °/s, damit ein von Hand ausgelöster Dreher nicht mit voller Kraft zurückschnappt. Die in Kapitel 2.3 beschriebene Drift bleibt dabei bestehen, sie summiert sich über die Flugdauer aber nur zu wenigen Grad und fällt gegenüber dem Nutzen der Richtungshaltung nicht ins Gewicht.

Diese Soll-Drehrate ist die Führungsgröße für einen inneren PID-Regler, der die vom Gyroskop gemessene und über ein PT1-Glied geglättete Ist-Drehrate ausregelt und daraus die Stellgröße für den Mixer berechnet. Das Filter dämpft hochfrequentes Sensorrauschen, bevor es den Differentialanteil des Reglers erreicht, der auf genau solches Rauschen empfindlich reagiert. Die drei PID-Regler sind unterschiedlich eingestellt, wie Tabelle 5 zeigt:

Tabelle 5: Reglereinstellungen

Achse	Kp	Ki	Kd
Roll	0,35	0,3	0,0007
Pitch	0,4	0,3	0,0007
Yaw	0,6	0,3	0

Yaw hat den höchsten Proportionalanteil, weil eine Drehung um die Hochachse hauptsächlich über die geringen Reaktionsmomente der Propeller erzeugt wird und deshalb einen stärkeren Regeleingriff braucht als Roll oder Pitch. Der Differentialanteil bleibt bei Yaw dagegen auf null: Die Achse ist ohnehin die schwächste der Drohne, weil sie nicht auf den direkten Schub der Motoren nach oben wirkt, sondern nur auf deren Drehmoment, und würde durch einen zusätzlich dämpfenden D-Anteil weiter geschwächt.

Der Integralanteil ist am Boden zusätzlich auf ±30 begrenzt und wird erst freigegeben, sobald das Gas einmal über einen Schwellenwert von 1350 lag; danach bleibt er bis zum Entschärfen frei. Der Schwellenwert liegt bewusst unter dem tatsächlichen Schwebegas: Läge er darüber, bliebe der I-Anteil im Flug dauerhaft gedeckelt und könnte einen bleibenden Trimmfehler nie ausregeln.

Aus den drei Reglerausgängen für Pitch (p), Roll (r) und Yaw (y) berechnet der Mixer die Differenz, die zwischen den vier Motoren bestehen muss:

Quellcode 1: Berechnung der Motordifferenzen im Mixer

```
int mix_FL =  p + r + y;
int mix_FR =  p - r - y;
int mix_BL = -p + r - y;
int mix_BR = -p - r + y;
```

Die Vorzeichen ergeben sich aus der Lage der Motoren zur jeweiligen Achse und aus der in Kapitel 5.2 festgelegten Drehrichtung. Ohne weitere Maßnahme würde dieser Mixer bei niedrigem Gas versagen: Sobald ein Motorwert unter das Minimum fiele, würde er hart auf den Leerlaufwert begrenzt, wodurch die geforderte Differenz zwischen den Motoren verloren ginge und die Drohne die Lage nicht mehr halten könnte. Diesem Problem begegnet die Software mit einem als Airmode bezeichneten Verfahren in zwei Schritten. Zunächst wird geprüft, ob die Spannweite zwischen dem größten und dem kleinsten

Mischwert überhaupt in den verfügbaren Stellbereich passt; ist sie zu groß, werden alle vier Werte gleichmäßig herunterskaliert, sodass die Verhältnisse zwischen den Achsen erhalten bleiben. Anschließend wird das gemeinsame Grundgas so weit verschoben, dass weder der größte noch der kleinste der vier Motorwerte über die Grenzen hinausläuft:

Quellcode 2: Airmode: Skalierung und Verschiebung des Grundgases

```
int mix_max = max(max(mix_FL, mix_FR), max(mix_BL, mix_BR));
int mix_min = min(min(mix_FL, mix_FR), min(mix_BL, mix_BR));
int mix_range = mix_max - mix_min;
const int allowed = MAX_THROTTLE - MIN_THROTTLE;

if (mix_range > allowed && mix_range > 0) {
  float scale = (float)allowed / (float)mix_range;
  mix_FL = lroundf(mix_FL * scale);
  mix_FR = lroundf(mix_FR * scale);
  mix_BL = lroundf(mix_BL * scale);
  mix_BR = lroundf(mix_BR * scale);
  mix_max = max(max(mix_FL, mix_FR), max(mix_BL, mix_BR));
  mix_min = min(min(mix_FL, mix_FR), min(mix_BL, mix_BR));
}

int base = base_throttle;
if (base + mix_min < MIN_THROTTLE) base = MIN_THROTTLE - mix_min;
if (base + mix_max > MAX_THROTTLE) base = MAX_THROTTLE - mix_max;
```

Erst danach werden die vier Werte auf die Motoren gegeben. Das Verfahren wurde nach eigener Beobachtung von Betaflight übernommen, wo dieselbe Technik eingesetzt wird, um die Steuerbarkeit bei niedrigem Gas zu erhalten (Kapitel 3).

PID-Regler und Mixer waren insgesamt der zeitaufwendigste Teil der Softwareentwicklung. Mehrere Vorversionen erwiesen sich als ungeeignet, teils zu einfach gehalten, teils so komplex, dass ihr Verhalten kaum noch nachzuvollziehen war, bis die aktuelle Fassung entstand.

6.4 Sicherheitsfunktionen

Bevor die Drohne überhaupt scharf geschaltet werden kann, prüfen beide Geräte unabhängig voneinander, ob die Sticks in Ruhelage stehen. Die Fernsteuerung gibt den Befehl nur frei, wenn Roll, Pitch und Yaw um weniger als 400 Rohwerte von der beim Einschalten gemessenen Mittelstellung abweichen und der Gas-Stick innerhalb des in Kapitel 7.2 beschriebenen Totbands liegt. Die Drohne prüft anschließend selbst noch einmal und lehnt das Scharfschalten ab, solange der empfangene Gaswert nicht unter 1220 liegt. Diese doppelte Prüfung schützt unter anderem vor der in Kapitel 8.5 beschriebenen Situation, in der falsch übertragene Stickwerte einen dauerhaften Ausschlag vortäuschten: Die Motoren laufen dann gar nicht erst an. Das Entschärfen ist dagegen jederzeit möglich, damit die Motoren in jeder Situation sofort abgeschaltet werden können.

Beim Scharfschalten springt das Gas sofort auf den Wert 1180 statt bei 1000 zu beginnen. Grund dafür war eine Beobachtung im alten 2S-Aufbau: Die Motoren liefen unterhalb dieses Werts nur unruhig, ohne wirklich zu drehen, und erst ab etwa 1180 setzte ein gleichmäßiger Lauf ein. Das Scharfschalten überspringt diesen Bereich deshalb direkt.

Wird die Drohne während des Fluges entschärft, sinkt das Gas nicht sofort auf null, sondern mit 0,4 Einheiten pro Millisekunde, was einem Abfall über rund 2,5 Sekunden entspricht. Diese Funktion entstand nach einem Vorfall während eines Testflugs: Ein spontanes Entschärfen in einer Stresssituation

ließ die Drohne ungebremst zu Boden fallen. Die sanfte Rampe soll dieses harte Abstürzen auch dann verhindern, wenn im Flug in Eile oder Panik entschärft wird.

Zwei weitere Funktionen lösen automatisch aus, unabhängig vom Piloten. Bleiben die Funkpakete für 500 ms aus, wechselt die Drohne in einen Failsafe-Zustand: Sie entschärft sich selbst und zwingt Roll, Pitch und Yaw auf ihre Mittelstellung, statt mit dem zuletzt empfangenen Stickwert weiterzufliegen. Weil dieses automatische Entschärfen denselben Ablauf wie ein manuelles auslöst, sinkt das Gas auch hier über die sanfte Rampe aus dem vorigen Absatz, statt abrupt abzuschalten. Die Drohne fällt bei Funkabriss also nicht einfach herunter, sondern sinkt kontrolliert.

Der Notstopp schützt vor einer besonders gefährlichen Folge des Integrator-Windups aus Kapitel 2.4. Ist die Drohne am Boden fixiert, etwa auf der Teststange oder im Gras hängend, kann sie eine Schräglage nicht durch tatsächliche Bewegung ausgleichen. Der I-Anteil summiert den bestehenden Fehler deshalb immer weiter auf, bis die Reglerausgabe an ihrer Begrenzung von 400 Einheiten hängen bleibt. Bekommt die Drohne danach Bewegungsfreiheit, etwa weil sie sich löst oder mehr Gas gegeben wird, wird dieser aufgestaute Wert auf einen Schlag wirksam: Ein oder mehrere Motoren springen abrupt auf einen deutlich höheren Wert, was zu einem unkontrollierten Kippen oder Überschlagen führen kann. Der Notstopp erkennt diesen Zustand an einer eigenen, tiefer liegenden Erkennungsschwelle von 350 Einheiten: Bleibt die Reglerausgabe einer Achse zu lange darüber, schaltet er die Motoren sofort ab (Signal 1000), bevor sich der aufgestaute Fehler entladen kann.

Zwei weitere Prüfungen greifen schon vor dem ersten Regelzyklus. Meldet sich der MPU6050 beim Start nicht über den I²C-Bus oder lässt sich ESP-NOW nicht initialisieren, nimmt die Flugsteuerung den Betrieb gar nicht erst auf: Das Programm bleibt in einer Endlosschleife und hält alle vier Motorsignale dauerhaft auf 1000, dem Wert, bei dem die Motoren stillstehen. Ohne Lagesensor wäre die Drohne nicht regelbar und ohne Funkverbindung nicht steuerbar, sodass ein blockierter Start in beiden Fällen die sichere Variante ist.

7 Fernsteuerung und Funkkommunikation

7.1 Hardware der Fernsteuerung

Das Gehäuse der Fernsteuerung ist wie der Rahmen der Drohne aus PLA gedruckt. Anders als dort ist es nicht mit Kabelbindern, sondern mit M2-Schrauben an den vier Ecken verschraubt (Abbildung 7 zeigt das CAD-Modell, Abbildung 8 die fertig aufgebaute Fernsteuerung).

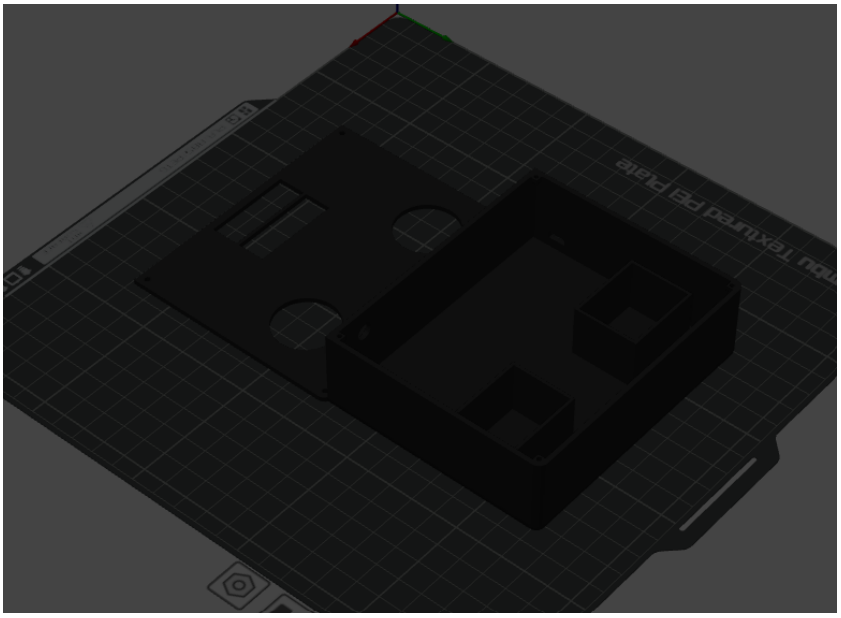


Abbildung 7: 3D-Modell des Gehäuses der Fernsteuerung



Abbildung 8: Aufgebaute Fernsteuerung

Die Energieversorgung beginnt bei einer 18650-Zelle, die über ein Lademodul mit USB-C-Eingang geladen werden kann. Vom Lademodul führen zwei Leitungen über einen Schalter zu einem Step-up-Wandler, der die Zellenspannung auf 5 V anhebt. Der Schalter sitzt bewusst vor dem Wandler statt dahinter: Ein Spannungswandler nimmt auch im Leerlauf einen kleinen Ruhestrom auf, und dieser entfällt vollständig, wenn bereits die Zuleitung zum Wandler unterbrochen ist. Am Eingang des Wandlers, also noch vor der Spannungswandlung, zweigt der Spannungsteiler zur Messung der Akkuspannung ab. Der Ausgang des Wandlers versorgt den ESP32 mit 5 V.

Von den 3,3-V- und Masseanschlüssen des ESP32 führt die Versorgung weiter zu einer ins Gehäuse geklebten Steckbrettleiste, die als gemeinsame Verteilung für beide Joysticks und beide Displays dient. Die Signalleitungen der Joysticks, der beiden Taster und der beiden I²C-Busse sind direkt mit dem ESP32 verbunden. Abbildung 9 zeigt den Schaltplan, Tabelle 6 die einzelnen Verbindungen.

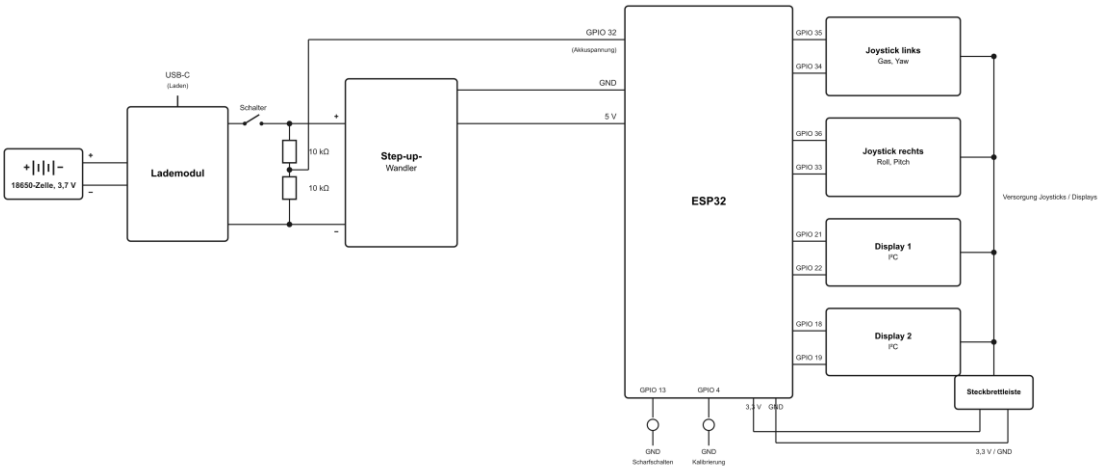


Abbildung 9: Schaltplan der Fernsteuerung

Tabelle 6: Verbindungen an der Fernsteuerung

Von	Nach	Funktion
18650-Zelle (+/−)	Lademodul, Zellenanschluss	Energieversorgung, gleichzeitig Ladeanschluss
USB-C	Lademodul, Ladeeingang	Aufladen der Zelle
Lademodul, Ausgang (+/−)	Schalter	Ein/Aus, vor dem Spannungswandler
Schalter, Ausgang	Step-up-Wandler, Eingang	Versorgung des Wandlers nur im eingeschalteten Zustand
Step-up-Wandler, Eingang (+/−)	Spannungsteiler (10 kΩ / 10 kΩ)	Messpfad für die Akkuspannung, vor der Wandlung abgegriffen
Spannungsteiler, Mittelpunkt	ESP32 GPIO 32	Analogeingang Akkuspannung
Step-up-Wandler, Ausgang (5 V/GND)	ESP32, 5 V/GND	Versorgung des ESP32
ESP32, 3,3 V / GND	Steckbrettleiste	gemeinsame Versorgung für Joysticks und Displays

Von	Nach	Funktion
Steckbrettleiste	Joystick 1, Joystick 2, Display 1, Display 2	Versorgung
ESP32 GPIO 35	Linker Joystick, Gas	Analogeingang
ESP32 GPIO 34	Linker Joystick, Yaw	Analogeingang
ESP32 GPIO 36 (VP)	Rechter Joystick, Roll	Analogeingang
ESP32 GPIO 33	Rechter Joystick, Pitch	Analogeingang
ESP32 GPIO 13	Taster Scharfschalten	interner Pull-up-Widerstand
ESP32 GPIO 4	Taster rechter Joystick, Kalibrierung	interner Pull-up-Widerstand
ESP32 GPIO 21 / 22	Display 1, SDA / SCL	I²C-Bus 1
ESP32 GPIO 18 / 19	Display 2, SDA / SCL	I²C-Bus 2

7.2 Software der Fernsteuerung

Weil ein Joystick beim Loslassen in seine Mittelstellung zurückfedert, kann der Gas-Stick keine absolute Position vorgeben. Die Software bildet daraus einen virtuellen Gas-Stick: Solange der Stick nach oben oder unten ausgelenkt ist, steigt oder sinkt ein intern gespeicherter Prozentwert mit fester Rate, in der Mittelstellung bleibt er stehen. Damit lässt sich das Gas wie an einer klassischen Fernsteuerung mit selbsthaltendem Hebel dosieren.

Die Rampe läuft zeitbasiert mit 200 Einheiten pro Sekunde, statt an die Zykluszeit der Hauptschleife gekoppelt zu sein, damit sie unabhängig von deren Laufzeit gleichmäßig verläuft:

Quellcode 3: Zeitbasierte Rampe des virtuellen Gas-Sticks

```
float step = THROTTLE_RATE_PER_S * dt;
if (stickDir > 0)      currentThrottle += step;
else if (stickDir < 0) currentThrottle -= step;
currentThrottle = constrain(currentThrottle, THROTTLE_MIN, THROTTLE_MAX);
```

Da die mechanische Mittelstellung jedes Joysticks fertigungsbedingt leicht abweicht, misst die Software beim Einschalten für jede Achse die tatsächliche Ruhelage, statt einen festen Wert anzunehmen. Ein Plausibilitätsbereich prüft dabei, ob der gemessene Wert überhaupt einer echten Stickmitte entsprechen kann. Genau diese Prüfung deckte während der Tests eine gebrochene Leitung an einem Potentiometer auf: Der gemessene Wert lag weit außerhalb des plausiblen Bereichs, weil die Achse durchgehend nur noch Massepegel lieferte. Seitdem sperrt die Software das Scharfschalten, wenn eine Achse unplausible Werte liefert, und zeigt auf dem Display an, welche Achse betroffen ist.

Als ausgelenkt gilt der Stick dabei nicht schon bei jeder minimalen Abweichung von der Mitte. Der Rohwert des Potentiometers schwankt auch im Ruhezustand geringfügig, und ohne eine Grenze würde dieses Rauschen ständig kleine, ungewollte Schritte am Prozentwert auslösen. Die Software erkennt eine Richtung deshalb erst, wenn der Rohwert die gemessene Mittelstellung um mehr als 450 Einheiten über- oder unterschreitet:

Quellcode 4: Erkennung einer aktiven Gas-Auslenkung

```
int stickDir = 0;
if (rawThrottle > center_thr + THROTTLE_DEADBAND)      stickDir = +1;
else if (rawThrottle < center_thr - THROTTLE_DEADBAND) stickDir = -1;
```

Erst wenn stickDir ungleich null ist, ändert sich der Prozentwert im nächsten Schritt. Das schmale Fenster um die Mitte wirkt damit wie ein Puffer gegen die natürliche Drift des Sticks.

Eine weitere Anpassung betraf das Loslassen des Gas-Sticks. Weil der Stick beim Zurückfedern kurz über die Mittelstellung hinausschwingt, wurde dieses Überschwingen zunächst als kurzer, echter Ausschlag in die Gegenrichtung gewertet und ließ den gespeicherten Prozentwert um ein bis zwei Prozent absinken. Eine kurze Sperrzeit nach jeder aktiven Auslenkung blendet dieses Überschwingen seitdem aus.

Die beiden Displays zeigen Verbindungsstatus, die Akkustände von Fernsteuerung und Drohne, den Schärfungszustand und die aktuelle Gasstellung an (Abbildung 10). Bleibt die Telemetrie der Drohne länger aus, wird das durch Platzhalterstriche kenntlich gemacht statt den letzten Wert stehen zu lassen. Ergänzt wurde später eine Anzeige der Gyroskop-Drift und der Sensortemperatur, damit erkennbar ist, wann eine erneute Kalibrierung sinnvoll ist.

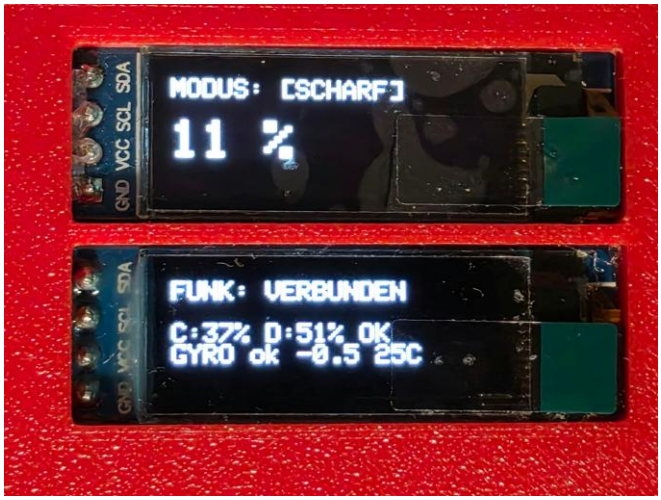


Abbildung 10: Anzeige der beiden Displays im Betrieb

7.3 Funkübertragung mit ESP-NOW

Bevor überhaupt Daten ausgetauscht werden konnten, musste die Verbindung zwischen beiden Geräten hergestellt werden. Zunächst wurde mit einem separaten, provisorischen Programm die MAC-Adresse des ESP32 in der Drohne ausgelesen und fest im Quellcode der Fernsteuerung hinterlegt, sodass diese von Anfang an weiß, an welche Adresse sie ihre Steuerpakete schickt.

In die umgekehrte Richtung wusste die Drohne zunächst nicht, an wen sie antworten sollte. Diese Notwendigkeit ergab sich erst später, als die Telemetrie mit Akkuspannung und Sensorstatus hinzukam. Statt auch hier eine feste Adresse zu hinterlegen, liest die Drohne die Absenderadresse direkt aus jedem empfangenen Paket aus und trägt sie beim ersten Empfang als Gegenstelle ein. Weil ohnehin schon ein Paket bei ihr ankam, ließ sich die Adresse darüber ableiten, ohne ein eigenes Verfahren zur erstmaligen Verbindungsaufnahme entwickeln zu müssen.

Das Eintragen dieser Adresse und der Versand der Telemetrie geschehen allerdings nicht in der Empfangsroutine selbst. Diese vermerkt lediglich, dass eine neue Absenderadresse vorliegt; die eigentliche Anmeldung der Gegenstelle und das Senden übernimmt ein eigener Task, der die Telemetrie etwa alle 100 ms verschickt. Damit wird die in Kapitel 2.5 genannte Vorgabe eingehalten, in den Callback-Funktionen keine längeren Operationen auszuführen, weil diese im WLAN-Task des ESP32 ablaufen [15].

8 Tests und Optimierung

8.1 Vorgehen und Sicherheit

Die ersten Tests fanden ausschließlich ohne Propeller statt, mit frei drehenden Motoren. Erst mit dem Beginn der Reglertests auf dem in Kapitel 8.4 beschriebenen Testgerüst wurden Propeller montiert. Der erste tatsächliche Flugversuch fand im Freien auf einer Wiese statt, einem Fußballfeld in der Nähe. Während der gesamten Testreihe wurde dabei Abstand zur Drohne gehalten. Ein Großteil der befolgten Sicherheitsregeln, etwa dieser Abstand, ergab sich erst aus den Zwischenfällen dieser Testphase selbst und war nicht von vornherein festgelegt.

8.2 Inbetriebnahme der Komponenten

Zunächst wurden alle vier Motoren am Testgerüst festgeschraubt und in identischer Reihenfolge verkabelt. Dabei drehten sich alle vier Motoren in dieselbe Richtung, was der in Kapitel 2.2 geforderten diagonalen Anordnung widerspricht. Behoben wurde das, indem bei zwei diagonal gegenüberliegenden Motoren zwei der drei Phasenkabel vertauscht wurden; welche zwei dabei getauscht werden, ist beliebig. Die endgültige Verkabelung ist in Kapitel 5.2 beschrieben.

Um die tatsächliche Drehrichtung ohne Propeller zu prüfen, ließ ein Testprogramm jeden Motor einzeln für einige Sekunden anlaufen, während eine gegen den Motor gehaltene Nadel die Drehrichtung spürbar machte. Bei den Propellern gab es ein ähnliches Problem: Die beiden Propellertypen für die beiden Drehrichtungen waren nicht eindeutig gekennzeichnet. Nachdem probeweise alle Propeller montiert waren, liefen die Motoren erneut einzeln nacheinander, während die Drohne sicher festgehalten wurde. Ein korrekt montierter Propeller drückt die Luft nach unten, was sich mit der Hand darunter als Luftzug spüren ließ; blieb dieser aus, saß an der Position der falsche Propellertyp.

Der MPU6050 wurde vor dem Einbau in die Drohne separat auf einem Breadboard mit dem ESP32 getestet (Abbildung 11). Ein Python-Skript visualisierte die gemessene Neigung als 3D-Modell auf dem PC: Kippte die Baugruppe auf dem Tisch, kippte die Darstellung entsprechend mit.

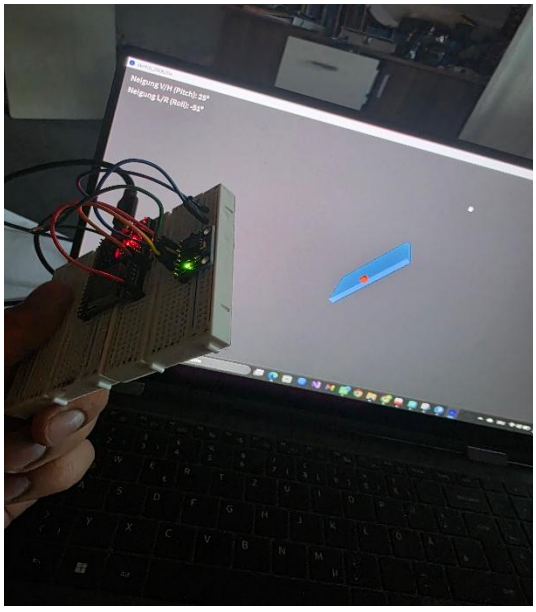


Abbildung 11: MPU6050-Test auf dem Breadboard mit Neigungsvisualisierung

Später diente meist der serielle Monitor zur Kontrolle, weil das genauer und einfacher war als der Vergleich mit der Bildschirmdarstellung. Vor den eigentlichen Reglertests wurde zusätzlich, ohne montierte Propeller, mit der Drohne in der Hand geprüft, ob die gemessene Lage korrekt interpretiert wird: Bei Neigung in eine Richtung mussten die Motoren auf der jeweils richtigen Seite hörbar stärker werden.

8.3 Iteration der Hardware

Der erste Test gegen eine aktive Auslenkung erfolgte noch mit dem ursprünglichen Rahmen und dem alten 2S-Akku. Wie in Kapitel 8.4 beschrieben, zeigte sich dabei, dass der oben auf dem Rahmen montierte Akku einen zu großen Hebelarm erzeugte: Die Motoren waren damit nicht stark genug, um eine Neigung aktiv auszugleichen. Nachdem der Akku stattdessen unten montiert wurde, konnte sich die Drohne gegen von Hand ausgeübte Störungen wieder selbst aufrichten.

Bei einem späteren Testflug fiel ein zweites, unabhängiges Problem auf: Obwohl der Akku noch deutlich Kapazität übrig hatte, ließ sich die Drohne nicht mehr abheben. Ursache war Spannungseinbruch unter Last, die Zellenspannung sackte beim hohen Strombedarf so weit ab, dass den Motoren nicht mehr genug Spannung blieb, obwohl der Ladezustand insgesamt noch ausreichend war. Nach Prüfung, dass ESC und Motoren mit einer höheren Zellenzahl kompatibel sind, wurde auf den 3S-Akku umgestellt.

Am selben Testflug nahm der Rahmen zudem einen kleineren, unkritischen Schaden, der mit bereits bestehenden Kritikpunkten zusammenfiel: Einige Stellen waren zu wackelig. Das führte zur Entscheidung, den Rahmen komplett neu aufzubauen statt ihn zu reparieren. Leitgedanke für den neuen Rahmen war, dass weniger Gewicht zusammen mit dem stärkeren Akku das Abheben erleichtern würde. Die Armlänge wurde auf die in Kapitel 5.1 begründeten 94 mm verkürzt, wodurch das Gesamtgewicht wie in Kapitel 4.2 beschrieben von 230 g auf 167,5 g sank. Insgesamt sank die zum Abheben nötige Gasstellung dadurch von 60 bis 70 Prozent auf 30 bis 40 Prozent.

8.4 Einstellung der Regler

Für die Reglertests wurde ein einfacher Testaufbau aus zwei Wasserkisten mit einer dazwischen gespannten Metallstange gebaut. Die Drohne wurde mit einem Kabelbinder in unterschiedlichen Positionen an der Stange befestigt, sodass sie sich jeweils nur um eine Achse neigen konnte, etwa nur nach vorne und hinten oder nur nach links und rechts (Abbildung 12).



Abbildung 12: Testaufbau mit dem ursprünglichen Rahmen, Akku hier noch oben montiert

Die Regler wurden nacheinander eingestellt, zuerst der Proportional-, dann der Differential- und zuletzt der Integralanteil, mit jeweils kurzen Tests nach jeder Änderung. Dabei zeigte sich das in Kapitel 8.3 beschriebene Problem des oben montierten Akkus: Die Motoren konnten die Lage gegen den großen Hebelarm nicht aktiv halten. Nach der Verlagerung des Akkus an die Unterseite hielt die Drohne einer von Hand ausgeübten Störung stand und richtete sich danach von selbst wieder auf, wie Abbildung 13 zeigt.

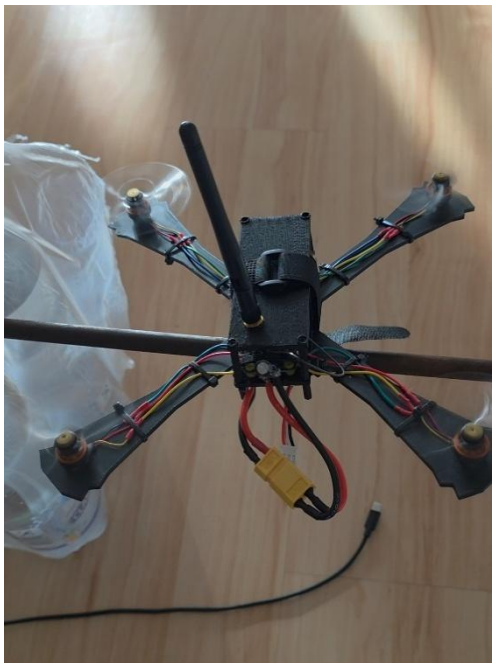


Abbildung 13: Standbild aus einem Testvideo, Akku unten montiert, Propeller durch die hohe Drehzahl kaum als solche erkennbar

8.5 Fehlersuche anhand von Messdaten

In mehreren Fällen gab erst ein konkreter Messwert den entscheidenden Hinweis auf die Ursache eines Problems. Als die Steuerung um die Rollachse völlig ausfiel und die Drohne dauerhaft nach links kippte, zeigte das serielle Log den Rohwert STICK r:1004, der sich trotz Stickbewegung nicht mehr veränderte. Ursache war eine gebrochene Leitung am Roll-Potentiometer des Joysticks; nach deren Reparatur reagierte die Achse wieder normal.

Ein zweiter Fall betraf das Gyroskop: Direkt nach dem Einschalten zeigte sich eine Drift von −17,3 °/s, weil der Sensor noch nicht seine Betriebstemperatur erreicht hatte. Abhilfe schuf eine Warmkalibrierung, bei der die Kalibrierung erst nach einer kurzen Aufwärmphase erfolgt, wodurch die Restdrift deutlich sank.

Der dritte Fall betrifft den Gas-Stick und ist bereits in Kapitel 7.2 im Detail beschrieben: Nach dem Loslassen sank der gespeicherte Prozentwert um ein bis zwei Prozent, weil das kurze Überschwingen des Sticks über die Mittelstellung hinaus als aktive Auslenkung gewertet wurde. Erst die dort beschriebene Sperrzeit blendet dieses Überschwingen zuverlässig aus.

Ein vierter Fall betraf einen Fehler in der Software der Fernsteuerung: Für Roll, Pitch und Yaw wurden zeitweise die rohen ADC-Werte des Sticks gesendet, statt sie zuvor auf den Bereich 1000 bis 2000 zu skalieren. Die Drohne empfing dadurch in Ruhestellung Werte um 1900 bis 2250 statt der erwarteten 1500 und interpretierte diese fälschlich als deutlichen Stickausschlag auf allen drei Achsen, obwohl die

Sticks tatsächlich in Mittelstellung standen. Der I-Anteil integrierte diesen dauerhaften Fehler auf, bis ein Motor auf Vollgas ging. Aufgedeckt wurde die eigentliche Ursache erst über das serielle Log der rohen Stickwerte: Es zeigte, dass der Regler korrekt rechnete, aber falsche Eingangswerte erhielt. Bis dahin waren wiederholt die Reglerparameter angepasst worden, obwohl der Regler gar nicht die Fehlerquelle war. Nach der Korrektur der Skalierung verhielt sich die Drohne wieder wie erwartet.

9 Fazit und Ausblick

9.1 Zusammenfassung

Im Rahmen dieses Projekts wurde ein Quadrocopter mit selbstprogrammierter Flugsteuerung entwickelt, wie es vom betreuenden Professor gefordert war. Sowohl die Drohne als auch die dazugehörige Fernsteuerung basieren auf jeweils einem ESP32 und kommunizieren über ESP-NOW miteinander. Die Flugsteuerung regelt Lage und Drehrate kaskadiert über PID-Regler, hält die Ausrichtung über einen Komplementärfilter aus Gyroskop- und Beschleunigungsdaten und bleibt auch bei niedrigem Gas über den Airmode-Mixer steuerbar. Ergänzt wird sie durch mehrere Sicherheitsfunktionen: eine sanfte Gasrampe beim Entschärfen, einen automatischen Failsafe bei Funkabriss und einen Notstopp gegen die Folgen von Integrator-Windup. Im Verlauf des Projekts wurde die Hardware einmal grundlegend überarbeitet, von einem 2S- auf einen 3S-Akku umgestellt und der Rahmen leichter und kompakter neu aufgebaut, wodurch sich die zum Abheben nötige Gasstellung von 60 bis 70 Prozent auf 30 bis 40 Prozent verringerte. Am Ende steht eine Drohne, die sich über die selbstgebaute Fernsteuerung kontrolliert fliegen lässt und deren Verhalten und Grenzen durch die eigene Entwicklung und Abstimmung jeder einzelnen Komponente genau bekannt sind.

9.2 Herausforderungen und Erkenntnisse

Eine erste Herausforderung war der hohe Rechercheaufwand zu Beginn, da zum Projektstart weder Bauteile noch konkrete Vorgaben vorhanden waren. Auch das Zusammenstellen der Bauteile und die saubere Verkabelung, bei der Anschlüsse und Spannungen sorgfältig geprüft werden mussten, damit nichts beschädigt wird, kostete viel Zeit. Beides hat aber gleichzeitig eigene Fähigkeiten in diesem Bereich spürbar verbessert. Auch die CAD-Konstruktion und der 3D-Druck, zunächst für den Rahmen der Drohne und später für das Gehäuse der Fernsteuerung, waren herausfordernd, wenn auch weniger als die übrigen Punkte.

Mit Abstand am schwierigsten und zeitaufwendigsten war jedoch das Schreiben und Testen der Software für die beiden ESP32, vor allem der Flugsteuerung. Dafür waren viele Testdurchläufe und mehrere neue Testaufbauten nötig, ebenso mussten viele Fehler gefunden und behoben werden. Dabei ging auch wiederholt Hardware kaputt, die anschließend repariert werden musste, etwa ein abgerissenes Motorkabel, das sich nur mit sehr feiner Lötarbeit wieder instand setzen ließ.

Auch bei der Hardware traten wiederholt Probleme auf, deren Ursache erst durch systematisches Messen mit dem Multimeter einzugrenzen war. Ein Beispiel: Ein Motor ließ sich zeitweise gar nicht mehr ansteuern. Zunächst wurde eine schlechte Lötstelle vermutet und nachgelötet, ohne Erfolg. Als Nächstes kam eine zu tief in den Motor eingedrehte Schraube infrage, was sich ebenfalls nicht bestätigte. Auch ein Test, ob überhaupt ein Signal vom ESP32 ankam, verlief unauffällig, sodass ein Defekt am ESC zunächst der wahrscheinlichste, aber auch unangenehmste Verdacht war. Erst eine Messung mit dem Multimeter zeigte, dass eines der drei Phasenkabel selbst mittendrin eine unterbrochene Verbindung hatte; nach dem Austausch dieses Kabels funktionierte der Motor wieder. Solche Fehlersuchen kosteten viel Zeit, haben aber die eigene Fähigkeit verbessert, systematisch nach der Ursache eines Defekts zu suchen. Ein Vorteil der vollständigen Eigenentwicklung ist außerdem, dass im Fall eines späteren Defekts eigenes Wissen über Aufbau und Verkabelung vorhanden ist, um die Reparatur selbst durchführen zu können.

9.3 Grenzen des Systems und Ausblick

Die größte Grenze des aktuellen Systems liegt in der reinen 6-Achsen-Sensorik ohne Positionsbestimmung: Ohne Referenz zur Umgebung driftet die Drohne, und ein Halten einer festen Position ist nicht möglich. Wie in Kapitel 3 am Beispiel von ESP-Drone gezeigt, ließe sich das mit einem Abstandssensor wie dem VL53L1X für die Höhe und einem optischen Flusssensor wie dem PMW3901 für die seitliche Position nachrüsten.

Auch die Reichweite ist begrenzt: Nach eigener Beobachtung während der Testflüge liegt sie über ESP-NOW bei etwa 400 Metern, eine genaue Messung wurde dazu nicht durchgeführt. Perspektivisch käme dafür GPS infrage, etwa um größere Distanzen zurückzulegen oder eine Position auch außerhalb der Sichtweite zu kennen. Eine in vergleichbaren Projekten übliche Erweiterung wäre außerdem eine Kamera, was allerdings sowohl bei der Fernsteuerung als auch bei der Funkübertragung einen deutlich größeren Aufwand bedeuten würde, da eine Videoübertragung andere Anforderungen an Bandbreite und Latenz stellt als die reine Steuerung.

Literaturverzeichnis

[1] PX4 Autopilot, „Basic Concepts," [Online]. Available: https://docs.px4.io/main/en/getting_started/px4_basic_concepts.html. [Zugriff am 15 September 2026].

[2] P. Yedamale, „Brushless DC (BLDC) Motor Fundamentals, Application Note AN885," 2003. [Online]. Available: https://ww1.microchip.com/downloads/en/appnotes/00885a.pdf. [Zugriff am 15 September 2026].

[3] Adafruit Industries, „Li-Ion & LiPoly Batteries: Voltages," [Online]. Available: https://learn.adafruit.com/li-ion-and-lipoly-batteries/voltages. [Zugriff am 15 September 2026].

[4] Grepow, „A Comprehensive Guide to Know LiPo Battery Voltage," 17 Juni 2025. [Online]. Available: https://www.grepow.com/blog/what-is-the-voltage-of-a-lipo-battery.html. [Zugriff am 15 September 2026].

[5] T. Luukkonen, „Modelling and control of quadcopter," Aalto University, Espoo, 2011.

[6] G. M. Hoffmann, H. Huang, S. L. Waslander und C. J. Tomlin, „Precision flight control for a multi-vehicle quadrotor helicopter testbed," *Control Engineering Practice,* Bd. 19, pp. 1023-1036, 2011.

[7] W. Moebs, S. J. Ling und J. Sanny, University Physics Volume 1, Houston: OpenStax, 2016.

[8] InvenSense Inc., „MPU-6000 and MPU-6050 Product Specification, PS-MPU-6000A-00, Revision 3.4," 2013. [Online]. Available: https://www.cdiweb.com/datasheets/invensense/mpu-6050_datasheet_v3%204.pdf. [Zugriff am 15 September 2026].

[9] M. Pedley, „Tilt Sensing Using a Three-Axis Accelerometer, Application Note AN3461, Rev. 6," 2013. [Online]. Available: https://www.nxp.com/docs/en/application-note/AN3461.pdf. [Zugriff am 15 September 2026].

[10] S. Colton, „The Balance Filter: A Simple Solution for Integrating Accelerometer and Gyroscope Measurements for a Balancing Platform," 2007. [Online]. Available: https://www.chiefdelphi.com/t/paper-the-balance-filter/168601. [Zugriff am 15 September 2026].

[11] K. J. Åström und R. M. Murray, Feedback Systems: An Introduction for Scientists and Engineers, Princeton: Princeton University Press, 2008.

[12] PX4 Autopilot, „Controller Diagrams," [Online]. Available: https://docs.px4.io/main/en/flight_stack/controller_diagrams.html. [Zugriff am 15 September 2026].

[13] PX4 Autopilot, „ESC Protocols," [Online]. Available: https://docs.px4.io/main/en/esc/esc_protocols.html. [Zugriff am 15 September 2026].

[14] NXP Semiconductors, „I²C-bus specification and user manual, UM10204, Rev. 7.0,“ 2021. [Online]. Available: https://www.nxp.com/docs/en/user-guide/UM10204.pdf. [Zugriff am 15 September 2026].

[15] Espressif Systems, „ESP-NOW,“ [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html. [Zugriff am 15 September 2026].

[16] Betaflight, „Betaflight, README,“ [Online]. Available: https://github.com/betaflight/betaflight/blob/master/README.md. [Zugriff am 15 September 2026].

[17] ArduPilot Dev Team, „ArduPilot Documentation,“ [Online]. Available: https://ardupilot.org/ardupilot/. [Zugriff am 15 September 2026].

[18] PX4 Autopilot, „PX4 Autopilot User Guide,“ [Online]. Available: https://docs.px4.io/main/en/. [Zugriff am 15 September 2026].

[19] SZ DJI Technology Co., Ltd., „DJI Mini 4 Pro, Specs,“ [Online]. Available: https://www.dji.com/global/mini-4-pro/specs. [Zugriff am 15 September 2026].

[20] Espressif Systems, „ESP-Drone, README,“ [Online]. Available: https://github.com/espressif/esp-drone/blob/master/README.md. [Zugriff am 15 September 2026].

[21] Espressif Systems, „Hardware Reference,“ [Online]. Available: https://docs.espressif.com/projects/espressif-esp-drone/en/latest/hardware.html. [Zugriff am 15 September 2026].

[22] AliExpress, „ESP32 DevKitC WIFI und Bluetooth Entwicklungsplatine mit 2,4-GHz-Antenne, ESP32-WROOM-32U,“ [Online]. Available: https://www.aliexpress.com/item/1005008949552110.html. [Zugriff am 15 September 2026].

[23] YSIDO, „YSIDO 1204 6000KV 2-3S Brushless-Motor für Rahmen von 100 bis 150 mm,“ [Online]. Available: https://www.aliexpress.com/item/1005004825662129.html. [Zugriff am 15 September 2026].

[24] AliExpress, „GY-521 MPU-6050 Modul, 3-Achsen-Gyroskop und 3-Achsen-Beschleunigungssensor,“ [Online]. Available: https://www.aliexpress.com/item/1005008604349573.html. [Zugriff am 15 September 2026].

[25] HAKRC, „HAKRC 15A 25A 2-4S 4-in-1 BLHeli_S ESC, DShot600, 20 x 20 mm,“ [Online]. Available: https://www.aliexpress.com/item/1005008561965143.html. [Zugriff am 15 September 2026].

[26] Gemfan, „Gemfan 3020, 3-Zoll-Dreiblattpropeller aus Polycarbonat,“ [Online]. Available: https://www.aliexpress.com/item/1005012260248254.html. [Zugriff am 15 September 2026].

[27] Tattu, „Tattu R-Line 3S LiPo-Akku, 11,1 V, 650 mAh, 95C, XT30,“ [Online]. Available: https://www.amazon.de/dp/B08535D7H4. [Zugriff am 15 September 2026].

[28] Teranty, „TERANTY LiPo-Akku 1500 mAh, 45C, 2S bis 6S,“ [Online]. Available: https://www.aliexpress.com/item/1005008972889776.html. [Zugriff am 15 September 2026].

[29] AliExpress, „Micro BEC 5 V / 3 A und 12 V / 2 A, Eingang 6 bis 28 V,“ [Online]. Available: https://www.aliexpress.com/item/1005010202743409.html. [Zugriff am 15 September 2026].

[30] GERUI, „OLED-Display-Modul 0,91 Zoll, 128 x 32 Pixel, I²C,“ [Online]. Available: https://www.amazon.de/dp/B0CT2QP43S. [Zugriff am 15 September 2026].

[31] AliExpress, „Zweiachsiges XY-Joystick-Modul KY-023,“ [Online]. Available: https://www.aliexpress.com/item/1005005496037039.html. [Zugriff am 15 September 2026].

[32] VISSQH, „18650 Lademodul mit Typ-C-Anschluss, DC-DC-Step-up-Wandler 2 A und Batteriehalter,“ [Online]. Available: https://www.amazon.de/dp/B0D3VCSNMS. [Zugriff am 15 September 2026].

Hinweis zur Nutzung von KI

Im Rahmen der Erstellung dieses Projektberichts sowie der Programmieraufgaben wurden KI-Tools unterstützend eingesetzt.

Das Tool diente einerseits zur sprachlichen Optimierung von Textpassagen und zur strukturellen Verbesserung des Berichts. Andererseits wurde es gezielt zur Unterstützung bei der Erstellung, Verbesserung und Fehleranalyse einzelner Codeabschnitte verwendet.

Alle durch die KI generierten Inhalte und Codes wurden eigenständig auf fachliche Richtigkeit überprüft, getestet und an die konkreten Anforderungen der jeweiligen Aufgabenstellung angepasst

Anhang

A 3D-Modelle

Die 3D-Modelle von Rahmen und Gehäuse der Fernsteuerung liegen als STL-Dateien separat bei.

B Quellcode der Flugsteuerung

Der vollständige Quellcode der Flugsteuerung liegt als Arduino-Sketch separat bei.

C Quellcode der Fernsteuerung

Der vollständige Quellcode der Fernsteuerung liegt als Arduino-Sketch separat bei.